

Active InferAnt Stream 013.1 ~ NoOrg is a Real Org

Source: [YouTube](#) **Playlist:** Active InferAnt Stream **Duration:** 1:44:00 |

Uploaded: Unknown **Transcript method:** auto_caption

All right. Hello everyone. Welcome. I'm going to begin this stream with a GitHub push to the Octave Inference Institute GitHub account no.org. So start this big push and then we are going to be in the game. All right. First, where we're headed, we are going to be designing cars for butterflies. And we're going to be doing that by orchestrating a ton of different kinds of different agents, LLM and otherwise. It's actually even a lot more general than that. So, stick around, find out, and if you're in the live chat, looking forward to your comments and questions. So, let's pull back a second. This is Active Infinance Stream 13.1. It's April 25th, 2025. I'm Daniel Friedman and the title of this stream is no or is a real or let us confirm that GitHub is still pushing. We'll reload it soon. All right. Big picture. This is part of some ongoing projects that are related to organizations documentation and multi-agent systems. This is open- source CCPsa 4.0. Now that the GitHub push has come through, we will reload the repo which was empty up until now and we see everything there. So let us start with looking at the outputs from the example. So where are we headed here? We're headed into extremely flexible, bottomup, vibecoded, open-source, customizable, multi-agent orchestration using an enormous repository of different documents and templates and tools. Um, as just one example, and I will look forward to people suggesting more examples in the chat, and we'll do a bunch of them. Where this is going to go is in the example number eight. At the top of the script is the convict. So this was about designing an eco-friendly transportation solution for pollinators to design a lightweight vehicle was the objective. There were some constraints related to toxicity and the target users are butterflies actually but conservation organizations as well. So just to sort of confirm this, I'll wait until somebody makes a suggestion and then I will modify the project config to be whatever other inventive scenario we want. And where that takes us is we get a few different visualizations. So here's a visualization of the whole workflow. First, the research unit does requirements analysis through LLM. send some messages out to the development unit, out to the community engagement unit, budget allocation, all these different rule-based and LLM based agents that are sending messages to and from each other. And that ends up getting uh okay by board. There's a bunch of different steps. We'll have different

examples. And then finally, it results in the marketing department doing some calls to some image generation services for what the final product might look like. So we can kind of pick um what type of organizational structure and flow we want and that is no or. So we will return to how the example works in a little bit more detail. First going to noorg.g. Again, anyone who writes a comment or an idea or a question in the chat, I will try to incorporate this. Okay. Why no? No. Or is not an organization. It's a metaorganization. A way of thinking about how organizations work. It is like cognition is to organization as metacognition is to no org. And that draws a lot from metagove. And that's going to come into play later in the stream. So part one, no org is not a real organization. It's not an organization. It is a template and a grammar and system for organizations. Kind of like how metacognition is about punch. But at the same time, no or if you break it up a little differently, not that it's not an organization, but N O knowledge as in new meets RG, which could be reormalization group, it could be role playing game. And it is a real org. This isn't just theory. No org is a functioning entity, a living system where different kinds of interaction generate tangible outcomes and structure. The no acknowledges the unconventional, the emergent, the resistance to rigid definition, yet affirms its genuine existence and operational reality. Or as they say, no organization, no organization, no organization, no organization. And this is not an organization yet. And perhaps it functions just as one. So after all, was it a real organization? People talk a lot about agents and agents in organizations. However, at the tactical and the operational level, this is coming down to among other things documentation, structure and process flows. We are at a point where with the qualitative understanding of this and the above plus a little bit of vibe coding it is possible to develop some groundup software packages and paradigms for flexible design development and deployment of agents in an open-source fashion that is agnostic of implementation structure function and process flow. Knowledge is relational. The unit paradox we will look at the units. Here is a quick look at the Obsidian documentation graph of what the unit structure looks like. So when you dispatch those messages, think about these units and all that they have to do. Here's here's operations unit, one of the core units. And here's all of the documents that are connected to this massive well ststructured at least in theory units repository. We're going to get there. What is a unit? What is nation? What is a tool? What is a finite? What is an infinite game? How do we play? How does the map redraw itself? How do we diagram out these different kinds of units and functionalities? How do we explore pattern over prescription and seek an emergent order? How do we come to understand that knowledge lives in practice? And this fundamental paradox of the empty center, the donor, Taurus, where and how do

duality and recursion come into play when thinking about the sacred geometries of organization of living systems? How can we use different kinds of interactivity to have this holistic synthetic understanding of structure for cyber physical systems? And where does that take us in terms of a cool like understanding of structure? Let us go to the examples after briefly looking at what the units are. So here is an obsidian representation graphical view of these different repos. we will go into the charter of the finance unit. So here we are looking at it in obsidian and here it is in cursor. So these are all files and there's a different amount in different ones of them but they all have some core files like the charter the index etc. Let's go to education. So here's the charter of education and it's written out in plain text and what that looks like in cursor in the education charter is these double bracketed links. So this is how obsidian knows that it is in an obsidian knowledge base and that gets rendered out like this. So education charter index of different documents within the education unit and this can go to an arbitrary level of detail. So let's just start up a new chat over here. Drag over education in the education unit. Comprehensively add sections and documents. for everything a education unit. So we'll let that process and the idea with this is they are human readable and there's ways to render in and out of obsidian knowledge bases but human readable forms that give these semantic interfaces that are going to come into play in several ways. So at one level just by having these files in the cursor repo when we then go and code something else somewhere else using prompt engineering then it's going to find some of this information relevance. So that's a very qualitative semantic proximity based approach. Like if we were going to go over to some folder and say write an education paradigm and then in the education folder there's already a curriculum and there's already these different kinds of things like AI principles curriculum. So education could host different information on initiatives, their their performance indications, different meeting notes that they've had within subgroups or different teams within their unit. So this is like an organizational structure and all of that is linked up in the unit directory. So here's the big picture of the unit. This is just one way to draw it out. So high level structure here's different organizational relationships. Let's look at it in GitHub units units are out. So in GitHub the double brackets don't render like they do in Obsidian. It's still there though. Still plain text. Part of the beauty of plain text. Organizational structure almost cool. It's like it's redacted. It's like it doesn't really matter which one they specifically are, but you can have detailed relationships with executive knowledge, corporate, infra, ops, product, tech, revenue, different templates and paradigms for strategy, tactics and operations, different information flow, different unit structure and the way that those link up. And then the dense linking in the

hypertext and in the obsidian form tables of information interfaces with different communities cross functionality. These can get spun out to the M degree and they have all kinds of information that is both at this uh kind of representation totally readable representable screenshotable by people also plain text versionable like you can version it in GitHub that's what we're going to be doing a lot of and then also it can be used for machine interpretability legibility of of documentation and all of these different kinds of features. So here is what's called 3.7 chugging away in the education unit. So here is updates [Music] happening updates happening in the education unit. So it enhanced the read me made a new structure for the curriculum. Now it's just writing and chugging along writing different materials for the education. We can add all all kinds of features there. So this project started as thinking well how how can we generalize across organizational structures? How can we here we are in the units. How can we look at different units and frame the relationship very generically? Kind of like a Linux kernel. The Linux distribution is not necessarily set up with your username and your password and your wallpaper. There might be some defaults or it might just be a placeholder. But then this can be rendered or deployed kind of like again a Linux kernel for organizations. Um, and it's an opportunity to flesh out some really exciting and interesting things in this generic. So here's another example of that. Within the security department, we have cognitive security, digital security, and physical security. Of course, cognitive being the most general of them all. And there's only a few documents in here now. But we can start to play around in the security unit. Oh, wait a minute. No, no playing in the security unit and start to explore what does that generic framework for security in an organization look like? And it's like as always the generic and the specific, right? genus and species generalizations and linear particulars. So we might draw and abduct or induct or deduct in different ways thinking about like this would be super cool for a research unit in general. What would be their focus areas in general? Or there could be a subresearch area like it could be the taste research unit. So these kind of nested and interacting units give a lot of opportunity to take the generalization right up to and probably a little bit beyond the point of being generic and end up with all of these different uh kinds of sub and fractal and nested interacting structures. So we will add a education subunit for language learning be comprehensive be comprehensively okay so that's kind of the unit structure and I believe that that is some of the utility and novelty in this situation which is that it's an obsidian open- source plain text approach to representing organizational structures in in the generic and in the specific case using Obsidian as a way to visualize this no org, right? Because it's not an organization. So all these different connections

and and people can look at how different things connect up and there's the unit directory, all the things that it's connected and and draws. Okay. So that is the unit component of this package. Push the GitHub update. Okay. Okay. back to the uh screen agenda which I'll just break down as we get through things. So the unit directory and the markdown relationships was what I began with was building this out and just continuing to say okay now add a new unit now add this unit now link these two units now add this unit now add more within this unit just what other units would be relevant just keeping going and going at all those different uh meta organizational meta governance type structures let's add a department of meta What? What's in units? Make a new unit with all needed documents and for meta documents about fractal and scale three and interacting. governance processes within and language learning subunit. We have a language learning folder. It's been one of the most effective augmenters of VI coding has been first off the multi-shot agent mode in cursor and then improved ability to have situational awareness in the file structure and being able to have new folders get made and uh make files iteratively all that kind of stuff plugging away on the language learning education unit is in process. Okay. So the units folder it's again it's uh pretty simple in concept and it can get rolled out to basically the nth degree of sophistication. You can get as many generic or specific policies as you want. Just keep going. You can continue to uh work off of the org chart, metaorg chart/module structure that is currently in the no org repo or I think you've probably seen enough even at this point just to go into cursor or whatever and just say okay build me a modular meta organizational structure roll out every single possible thing that could be relevant for XY and that that'll kind of build you what you not at the right level of detail and and you know tuning and tweaking along the way. Okay, back to the again. Third time to return to this. But I started with just the Obsidian repo. There was no code in this repo at all. It was just about well if you consider what you consider a code what you consider a code uh only in the units repo uh just exploring and I thought okay well wouldn't it be pretty cool to be able to start to deploy custom configurations of agents within those units and that is where it went. So there's a few different other folders to look at. Agents has a lot of information. All of these documents, ridiculous numbers of documents. So here, this Obsidian window is only looking at the units. Now, this one is pulled out to the broader no.org. So they're both kind of mutating as the metaggo files come into play. So here's the broader organization. slightly different, much larger um network, lot of more standalone files because uh outside of the units directory, there's less obsidian linking here in the uh agents again top level folder. There's all these and ranging from the these some of these are obsidian link.

They're they're more heterogeneous. Let's look at it in let's see. So this is like why putting stuff online makes a semantic impact even if it's hard to get attention of people in the moment. It's like all the work that Metagood and people using the term have done, some of it gets slurped up, used as training data and that supports later through this kind of airlock of not really citing them. Sometimes a specific citation or URL or person might pop up, but it's uh using the semantics of how meta governance was discussed by the people who discussed it. That's how symbols and words work. So here's a charter for a meta meta governance unit. And then if people want to configure their own organization as opposed to this non-existence organization that we keep on talking about with without this or with it, you can have knowledge management. You can have a legal department or or not. But it's configurable. That's kind of the cool thing. Okay, back to looking at the agents see it's a lot of there there's some code in the agents folder but not not not that much of that here are a large number of agentic semantic coding. So this is like at the meta agent implementation level all these different kinds of schema uh just at the conceptual level like if we want to talk about how do we tune performance in multi- agent systems just some pseudo code some different ideas some different patterns how do we think about autonomous learning How do we think about learning architectures, learning systems, different ways? It could be pseudo code. But this is not meant to be executable directly. But again, where's all of where's all this going with the V code? Natural language is code code along all that. And so these kinds of plain text which are now happily publicly available in this open source repo it it plays a role. So if someone says change tracking and make new changes yellow, that's going to move the needle in some little way into who knows how long later and someone says make a visualization for change tracking and that yellow is going to put its little syntactic finger on the scale. So that's what's happening in the system agent system folder is different kinds of looser semantics using natural language. But again, heavily human interpretable and heavily machine readable for all these kinds of qualitative interpretable elements of agents. There are guides, just a few of them, but they are guides to different situations where you might want to do action line. So, here's like a swarm implementation. Some of this refers to some real code, some of it doesn't, but the whole point. And what we see is you just say, "Okay, build it out where it's missing. Use this pattern and and go from there." And that will basically get you where you need to go. So guides, examples. Let's look at it in both places. Here are again markdown files of different examples of multi-agent behavior. So part of it is like why why state the office? But it it gives it if you want to just say go to the continent or go to this region or go to this city block or go to this

nanometer. There's multiple levels of semantic precision and it's not that more is simply better. So actually having a really elegant small concise way to be like when we talk about task execution behavior here's broadly what we're talking about. We mean that this is what happens or when we talk about learning this is what we mean. Those are helpful and then more technical details can always be scaffolded and layered in. Maybe there's programming languages, coding agents, all this kind of stuff we don't even know about. I would highly believe that. So why not work at a level like a meta organizational level or a meta programming language level or a metal linguistic level that can always be deployed in different ways. What do we mean by coordination behavior? uh in the communication folder just a few documents about communication specifically. It's like you can keep on growing these documents and then it's okay now let's have a subfolder for just monitoring. Let's have a subfolder for just task execution. Keep on growing out and and watering and improving these documents and modifying them as they go. and architectures which give a few documents that are related to architectural concepts in agent design like cognitive models. So the next time that there's a new paper or a new new social media post or some other kind of cognitive model, it can just get plugged right in here. So it's kind of like a new restaurant opening in a region. it can already be linked in to what is the region, what is the kind of food, it's a restaurant, all these background pieces of information that already bring it into this very interoperable system. So that's the agent archive has a few different uh prior tests. Let's see if any of them are relevant. This one was a multi- agent sorting algorithm just using this swarm dispatch paradigm to to do some sorting on different uh numbers. This one was a test suite that was written to do a lot of the functional testing for uh multi- agent systems. So looking at task performances and then it got up to doing the towers of Hanoi. So this is like a a team of agents that were spun up and spun down that were playing this towers. So people see that that's in the archive. All right. Then there's dist. So this was another uh approach to writing out different kinds of multi-agent systems. different ways. There's multiple different frameworks. Obviously, that would be saying least that there's many different ways to do multiation modeling. So, so these are just showing that there's a few different ways to have in this case JavaScript agents that have different kinds of capacities, different kinds of abilities that could be augmented or or the semantics could be transferred to all different kinds of settings. That's the disc. And this is a uh uh message passing architecture that that actually does have some function, but it's not the main one. And uh last before we get to the examples of this TypeScript code is the docs folder, the metadox, the paradox, the the docs of docs. Here is is a

um vast number of folders of documents that some of them have a ton of information, some of them have just a little bit of information and they are just another way to include semantics related to these topics. Pretty interesting to think about like where and how is the folder structure just the regular computer nested folder structure sufficient for conveying the kinds of structural and infrastructural and superstructural information that we want to convey and where do we want to use like graphical databases other methods. So, we won't look through all these documents, of course, but they're all up there and they come into play when you use curso carefully, then continue to add links and relevant metag stuff. So the docs folder has all different kinds of relevant documentation that can be brought into play in in different ways. So it can again be used at that qualitative level of just like here is AGI safety. Well, then if you type in something related to that or something happens related to that, then whatever is in this file is going to come into more play than it would have otherwise. Uh there is a bunch of sections. Uh let's see if there's any other ones. Poor documentation. That's the documentation on the documentation on the documentation. Different ways to do automation. The git is actually a real one. Fractic git is a fractal git program. So just tucking away. It's kind of like you're looking through a library, a hyper library of documentation and sometimes there's a little bookmark, sometimes there's a little pheromone trail. Other times it's like, huh, that's a whole functional piece of software. So in this case, the fractit system as inspired by proof of course is a fractal git coordinator that spins up and down and makes different kinds of git operations on a fractal git structure. So that kind of extends the git concept to these nested quantum superpositional repos. But it could be anything. But it it actually does have some utility. So being able to make repos spin up, spin down, clone, merge, all those different kinds of operations on fractal git operations. You give a lot of flexibility because again though it's probably or possibly not the most high bandwidth or or high utility representation structure to use the computer nesting system. And even though it might not be the exact set of commands that are the most useful in all possible universes like the default Linux ones, there's a ridiculous amount of training data on these kinds of things. So using folder structures and git play text you can get really far and do a lot. So metagove still chugging away. see if there's any other types of documentation that are different uh integrations with messaging platforms or could get integrated to uh matter most matter bridge be able to to bridge across different messaging platforms some related sections to the units lot of maintenance information. So what what at this meta documentational level what are good documentation maintenance practices that can be specified whether you're talking about documentation for building a car,

driving a car, whatever it is. this is the meta all of that and then today or tomorrow you can send a person undergrad code agent whatever and say go to the maintenance folder and do da da da and all all of that screenshot management we've been in some situations where the the proven screenshot was was make or break it was sink or swim so having the screenshot management on point it's like I've never seen it written out that way, but I could see why it matters once it was written out a way. Meta documentation in the meta documentation is actually quite a large section with information on the information on structure and information helping provide some meta documentation on this docs folder. different metrics. So then when a prompt is used like make it secure or make it high quality or make it performance those can be just filled up with the make it successful, make it valuable. Those are going to be just drawing on the base semantic distribution of whatever model you use plus whatever indirect context you have in the codebase or scope plus whatever are in these documents. So if you super emphasize that quality means elegance then that is going to move the needle in that direction. Let's just see if there's any other then we'll go to the examples. user interface troubleshooting training. Just open these up, see what's there, and then we will close Obsidian and work more with the code examples, get back to that butterfly call. So meanwhile, if anyone watching has a suggestion for a uh a uh something that they want invented through this multi- agent orchestration or a new setting, then check. Yeah, prompts. This is just kind of there's there's a lot of ways to do prompt management of course, but here the prompt folder. So then when when you write a prompt or even not do good knowledge management and as as usual mermaid diagrams coming in super clutch very few tokens so very easy to make a difference on a text model and it renders plain text etc for for high information throughput for for people and for visual Accessibility prompts folder quality assurance research search security different security concepts different ways to to discuss it setting up software standards strategies systems system science more systems templates I mean I it just it it is kind of like just to continue to play out. What does it look like when we keep on nesting templates like okay well we could have templates about meeting notes so that you can get into more and more levels of detail with templates on on just more and more and more or not just use the base probabilistic distribution of the language model. But if a template is in place, you can investigate it and it it testing tools training troubleshoot. All right, that will be it for the Obsidian part and we will look now at the at where we got here. Okay. So, I'm going to focus on the experiment number eight and nine. Eight is going to be a uh 7, 8, and 9 are are uh some of the more uh sophisticated ones. seven is uh and and all this goes into the output folder. Oh,

all the the source code for all of these examples is in SRC. So here's where there's a ton of information, utilities, uh there's a documentation folder inside of SRC, of course, core methods, different monitoring, messaging, different patterns, different typescripts, uh and some agent types, which can be up to the we'll probably not go too deep into SRC, but let's look at the examples now. Uh so in the multi- unit LLM flow seven which will just kind of pass through because uh eight is a little bit bigger. So in case this up. So here's this was an orchestrated flow. the these things are going to happen in this order and this is the summary of of what happened. So the the the initial prompt that was given to the to the innovation unit was uh a machine for counting and behavior and then this cost it looks like 67 cents. Not not sure if that was accurate. Probably less than that. Um and that summarizes the entire process. uh it is first sent from the innovation needed to assess the idea then the the the uh generation of ideas. So here's the idea generation response. But we'll we'll look at this in. So prompt is a fixed prompt and then it gets back. Okay, here's ant scope, ant monitor. Here's different ideas. Ants site, ant secure, ant connect that gets sent, the idea generation gets sent to the market research security assessment and then finally to the board of review. So let's look at the board review. The ANSCO project aligns well with our organization's strategic focus on using technology to address environmental concerns etc etc that is then refined with feedback at the innovation unit. There's a technical and a brand planning and then finally the final project plan is written out. So all of this was just autonomously orchestrated based on this fixed organization topology. So final project plan, it's an executive summary with all of the information from the different sections. So that's seven, example seven, the multi-unit flow. Okay, eight is the the one that uh is brings in a few other features. All right. You see person? Okay. Uh so as shown a little bit earlier what this script is going to do is demonstrate combining multiple agent types in a single workflow. There are some LLM calls there are some rulebased systems and there's also image generation with different kinds of coordinations. So let us modify this section. Well, first anyone watching live, write your idea for a project or the context for a project and meanwhile we'll just look at the butterfly car design to look at where we're going to get which is all the intermediates are safe. So here's like the architecture designer. The the system prompt is you're a systems architect system generic prompt. And then the user prompt is design a lightweight aerodynamic vehicle optimized for butterfly transportation in sync. And then the output from the architecture designer is this is what this is the concept. It's in adjacent so it's wide but it it feels worthy now having GDPR for butterflies using different kinds of access control and encryption for the um images folder are where the final prompts are created and

then sent out. So here's like the butterfly card, here's a logo and here's a systems representation. So let us uh make another concept just to confirm that it's doing it from scratch using control K just to modify it in line. This is also quite long file. This is over 3,000 lines of code. All right. the update this. So it is designing a physical crypto currency hardware wallet that must be invisible and waterproof for children. So now it's called stealth kids crypto warm and all I prompted here using the cursor was prompting to change these projects. So it could be different domain, could be different user group, different constraints. You know, it has to cost \$0 or whatever it is. Then over on the right hand side, I'm going to just ask cursor to run 8. So it will generate up the codes running. All right. So meanwhile, while Walt 8 is running over there, we're going to look at the um example N. All right. So, example nine, here's where the uh output is going to get to. So this is the executive summary. This report presents the results of an active inference partially observable composition process model used to analyze temperature changes throughout a narrative. The model was designed to extract temperature information from the story and determine optimal actions based on the inferred states in the environment. So, this has information on this. Let's put that here. So, here's beliefs belief updating about temperature. Some of these can can be fleshed out. Um, but the way that this one works is that a story is defined in natural language in the prompt and then first an LLM is going to be called to say what do you think the temperature was at each beginning, middle and end of the story and then the outputs of that structured outputs from that LLM end up getting taken as observations for a palm DP that then runs an active inference algorithm. So this uh is uh more of just a two-step uh program eight running in the background. So this one let's also do a little modification. So here in this story it was crisp in the morning then the sun was high. So it's kind of indirectly saying it was it was the weather was perfect for hiking neither too hot nor too cold but in the morning it was quite cold outside the temperature continued to drop. So, it kind of went from like medium to cold, from from cold to medium to very cold. Right. A new story about an egg crying as the in the room changes. Okay, so eight is running over there on the sideline. And so you can Oh, interesting. Interesting story. Let's here just run. There's some document chunking and splitting techniques. So here we are looking at output. Did we save it? Okay. Oh, it's still working in the butterfly situation. This is another freshly another a different butterfly. Wow. It's like a spaceship full of butterflies just like it should be. And we'll see the third one just appear. Butterfly. All right. It's just cuz I didn't save it. Not saved. Great. We just changed the part again. So that that'll show up in the folder as Stealth Kids crypto wallet. Okay, while that's going. All right, we did the GitHub push. We looked at the outputs

from the examples. We looked at the butterfly car design and now we're making the stealth crypto kids model. Looked over the sections of no or this is not an organization. You know what is it? Reviewed the unit meta architecture. talked about the documentation. All right, there's two more sections that uh few more sections. First, I'm going to look at cognitism by my colleague John Ash and explore what happens when we input the concepts of cognitism into this framework. Then we will go to Metag and pick one of their projects, possibly the Constitutions, possibly a living manifesto for online spaces and and see where that goes in this no org set and then look at any comments in the live chat and like if somebody wants to suggest another cool product to design or or some other funny thing. So here's now here's new active output. So let's look at this report. Okay. Similar think people get it. Uh this is a very useful pattern though what's happening in nine which is taking in natural language and then having LLM do some kind of inference on it like tell me how many characters are in the story or how many facts support this versus how many facts support that or what temperature do you think it is or how much do you think that would have cost all those kinds of questions. Not that it necessarily returns an accurate understanding, but first off, you can sample it a bunch of times at different temperatures, and that enables you to get a really good inference on the lat space of the LLM. So, you can say, what GPS coordinate do you think this picture is taken at? And so you can resample that a bunch of times to get the underlying distribution of its beliefs about the GPS locations of that prompt given the context. And that doesn't mean it's accurate, but it does mean that you have made an accurate measurement of that probabilistic system. Okay. Now, now we're outputting the folder stealth kids crypto wallet. So let's look at the outputs here. Okay. Children's 6 to 12. Secure, easy to use. Invisible, waterproof. How? How? Here's an interesting question. How will it be represented if it's invisible? Children's resiliency, water resistance, security features, security features, digitalization, digitalization. Okay. D and C water. That's kind of cool cuz it's waterproof. That makes sense. Okay. Okay. Imagine a traveling magic show. I'm not sure we're going to work with this design firm again, but it at least at least it looks kind of cool. Maybe the whole thing is it's like it's invisible. Awesome. School programs abstract. Okay. Um so example nine is uh again a useful pattern for those who are keeping school of using LLM calls to output structured basically as a transformer as as they as they are between natural language or structured information and some kind of output that you know what kind of output you're going to get and then you can use that as an input to some kind of basing inference or explicit probabilistic model. That's super useful. Tons of ways to use that. And then in eight is this other pattern of just using sequences of uh

sequences of orchestrated. Let's look at the Stealth Kids Crypto Wallet So this was the the designing that invisible waterproof wallet. So here was the orchestration pattern and in this situation the orchestration patterns fixed within the example but you can imagine creating a situation where it's dynamic. So let's just try that. say the functionalities of these two. Let's just focus on just the LLM orchestration of of of this example. Write a new example 10. where there is a config at the top. So that the user inputs natural language for what kind organization flow they want to see. We'll see if we can oneshot this. This might be a few steps. and first M LM call assesses that and returns whatever structured format you need so that that organizational flow can be implemented using the approach Similarly with all intermediates saved and the comprehensive org chart HTML visualization with the org chart as prepared after that special first up. Then take in a prompt from second part of the config block another variable for what custom or C. Okay. Let's look at caucusism and let's flip that shuffle. Okay. Due to the size of the margin, it will be left as an exercise for the reader to learn more about what John Ash is up to, what toxicism is about. Medium is a good place to see his work. I'm just going to see if copying this one glossary document is enough to make some kind of cool example. So, making a new folder sizer. Let me see if there's any other blog post. Let's just try it with just the wash. Okay. So that's 10. in this folder templating off of the functional eight. Write a creative, relevant, useful implementation of the cognitism concept. Make it very relevant and flexible and with focus visualizations and animations. Start with CS background. If you want to make a Python file in the folder or more analysis and visualization and animation, please do so. Okay. All right. So here comes 10 for dynamic board generator. And then in the third of the three possible shots that are interesting, let's look at what we could do for that. Okay. I don't see where the actual document is. Okay. Constitutions of web 3 might be a good one. So this is a work by Seth Bray and Josh Tan and I know they have a GitHub rep. All right. So here they curated some constitutions from different dows and different groups. So just as an example, let's if if somebody wants to pick a uh DAO, let's do the DA. Let's do Lex D and say we'll have a good legal constitution, right? I'm going to do something pretty similar. Examples, new folder, lex file, let's style on state. All right. Now I'm going to use the same prompt with box templating off of eight. Write a creative useful implementation of how lifestyle works. All right. So we got cognacism plug here. Lex down here. All right. In terms of tool use, cursor 49.6 mainly using claw 3.7 sonnet. Uh, I'm also using Gemini 2.5 a little bit. There's some things where it's doing well, but there is a few other times where it was just acting up. Using Obsidian for the markdowns. I'll push a GitHub update. Let's see if there's

any other Madag project that we can go. All right. Do something with contact. All right. All those are playing. We'll prepare prepare the fields. We're call home. I'm just going to copy just control a just contact website put in just a few unstructured pieces of information. And I'm walking All right. At the bottom there. So obviously this information can be structured better just to show that perhaps it doesn't even matter. Okay. Getting some errors with calling tools. We'll see if that works itself out. Otherwise, it might just be trying to do too much in one bite. Okay. At this point, I will uh while these slowly turn, I will mention that there's different ways to support the work. You could donate at [donate.active inference.institute](https://donate.activeinference.institute). You could get involved with the open- source development. You could use this and reference it. You could fork it and develop it or contribute to the main repo. Why not? We will be having a upcoming series on multi- agent systems design starting next week and that will be a one month design phase about uh open to all people who want to participate at the institute. no specific background. The meetings are going to be at 16 UTC on Tuesday and 19 UTC on Wednesdays and it's going to be first a design phase of one month to spec out like what kind of a multi-agent system are you interested to be developing? Uh what kinds of agents, what kinds of information environments, what kinds of flows, all those types of questions. And then in June, we're going to move it into an implementation phase. So, as kind of shown here, it's easy to get a sloppy design and implementation with basically just kind of natural language prompting or just using. Um, but I think over the coming month we're going to have a lot of really exciting ways to work at that specification level and then have strong ways to go from that specification to a high reliability very well implemented system. So, I would uh encourage anyone who's kind of peaked by this stream to get involved that way or if it's after then just probably still all good. Um, yep. Getting a lot of errors with me calling the edit file, but at least sometimes it's getting through. It's just having to do it in shorter sections. Um, one more piece that doesn't require the LLM. This is a paper that I came across, notation as a tool of thought by Iverson from IBM research. So, a few few quotes here. By relieving the brain of all unnecessary work, a good notation sets it free to concentrate on more advanced problems and in effect increases the mental power of the race. A non and the quantity of being compressed into a small space by algebraic signs is another circumstance that facilitates the reasonings we are accustomed to carry on by their date. How relevant. And this uh paper goes into some pretty exciting directions discussing notation and about ways to have information compression and operations through notation. So kind of like girdles orbach a notation scheme is developed like loop and loop and it's used to carry out a few different kinds of things like imaginary

ideas. But now we can make these notation systems that are just able to blur the line between the symbolic and the probabilistic. And uh some of these things would take a long time to understand manually but it would be possible to put this into an LM and uh develop implementations based upon that. Like this would be literally like learning a whole new language. But we could just put it in a file and say okay given that this is the language that we speak let's let's just see what happens notation. This is going to be called tool uh dot notation. Okay. All right. Run the dynamic or generator and fix it if anything errors. Okay, in place now in place. Let's see if the dynamic generator works for 10. So, of course, this is all happening real time, unscripted. Neither I nor the machine. So, it's more of an experiment see will it work, right? available in notation here. I'll use the control K to use an inline keeping all the information exactly structured this perfectly and we could pick a different model. We could use 04.04 04 mini is also pretty useful. Awesome though. It's just a little bit slow in this document. write up a bunch of well annotated examples of using this notation guys on using it in different places. All right. All right. But once it gets started, then it will then it starts to fix some of that. Okay. Yeah, several errors in that first draft, but now it is going to at least continue to go. Okay. I don't know why we did this. Yep. All the levels are a little slow, right? They are getting the job done. Let's try something different. Nice. takes a few sometimes, but yeah, it could be faster but at higher LLM cost. Okay, here we go. So, Gemini coming through. All right. So, structured. Wow. Okay. Using the control K in line. right. Something happened. Okay. How do you use? So more examples just sort of just um kind of respect and uh weed back with past. These are all like kind of elaborate notation for thinking approaches. all the stuff GNN AOS cerebrum all that kind of stuff is like using function with notation in these stock ways. So, agreement engine, a lot of similarities. Uh, more of a focus on agreement. Looks like it hasn't been updated in a couple of years. Um, but similar concept, different kinds of organizational flows and uh could be could be revisited in our five era. community rules. Similarly, good to be uh play Okay. Calling the system here. This file is 2030 appropriate. That's now 23. Okay. Visualizations for conscious. I will take a sip of water. See if anyone writes any questions or ideas in the chat and vote that these finish. right. Lex style is done. Run to confirm. Okay, update the dynamic exam. All good. Let's switch over to Gemini. Okay, going faster. Let's just do a roll continue and hands. Okay, maybe it won't happen in this stream. I don't want to stay on too long unnecessarily, but I will uh have to partial function versions stop. Ah, and then I'll get here. Okay, maybe there actually. Okay, it's the epistemic slot machine. It's like one more. So close. Push an update. Oh, look at the chat

update. LS added to the list. Okay. All right. Maybe we will get there. That's always part of the company. ignore and continue. That's something's happening there. Something is happening here. Okay. All right. We could not. All right. Awesome. All right. Incredible. Actually incredible. All right. Let's let's do a different organizational topology just to confirm the value. Let's now run. Here we go. So dynamic or generate. All right. So that was okay. Organization description task. We are updating this to be an organization that is literally shaped like a skyscraper. It is like a skyscraper shaped like a lollipop with a bunch of vertically nested layers and on the top layer it is hub and spoke with six connections. The task uh for the organization is actually appropriately to produce lollipops. I modified the quantity. run it again. All right, we'll see if it actually got there. Okay, Lexile is working. Never mind. Come on. Initialization. Okay. Okay. Proposals visualized it. So, um it is hanging here. Fix that and run it again. You changing please and not again. All right, let's look at the second dynamic or see if we got the lollipop code. Who represents the lollipop guild of of the lollipop for the lollipop? They succeeded. Fantastic. Okay. Okay. I wouldn't say that's exactly it, but pretty awesome nonetheless. Let's just see if we can get the palm attack going. in chat. I'm going to call Ter of use and elaborate on that. So it is a new example comprehensively representing home tech and organizational ergonomics positional. Okay, let's improve it. Please summarize exactly what to do. Okay, copy that. Okay, Tucker, thank you for your messages. Yes, using cases. Absolutely. Let's do that. Okay, we'll do that as language. case. organizational case comprehensively write up and obsidian late documents about how linguistic concept case applies to loyalty agents and multi- organizational unit operations make it incredible and relevant and all right Yep. Everybody wants to talk about organizational grammar, but how many want to talk about linguistic peace? Looks great. Could use cerebrum. Of course, it looks pretty still right there. It looks like it's installed here. Assess and improve it and re run Okay, let's see if it looks good. Yeah, but if anybody in these last few minutes has any uh comment they want read on stream or just an idea. Okay. Okay. Lex now is running. Oh, nice. Just going to leave that one there because it's going too deep into the core mode and we'll let these play out. I'll just give another minute if anybody wants to make a comment or question. The total cost of the API credits to OpenAI was about \$4 for for everything today. could have been could have been cheaper probably a few different ways, but all of the work to the cursor was of course a lot more than that yet probably much cheaper. Awesome. That's now Maybe it's generating a very long file. Okay. Oh, there it is. Let's see if we can get it. Okay. In closing, let's do in docs dialogues. We'll do move on first. \$4 true in the chat. \$4 API cost for all of these examples that I'm

running. So like many dozens including a lot of the image generation which is probably takes up a large amount here right a synoptic panoptic all good all beautiful all truthful truly hilarious who's on First style Dialogue regarding all the examples down still Little Okay, I'm going to call it there, but as soon as the burst comes through, right. See that? We'll just push the last update. Finishing the stream. And then we'll look at the dialogue docs dialogues. Okay, there it is. All right. Hope this was uh useful for some, interesting for some. It's definitely been fun to work on, and I'm appreciate if you've watched this far and attended to it. It's uh pretty fun. So, looking forward to people's comments and contacting me and however. All right, till next time.